Computer Science Internal Assessment

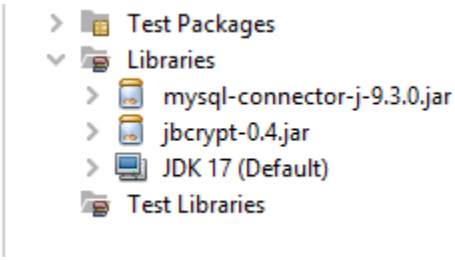
IA Project: Student Management System

Table of Contents

Imports and Dependencies....2
Techniques complexity used....2
Normal Framework....3
Password Encryption....5
Import multiple packages....6
Libraries....6
Images Package....6
Image handling....7
Static & Non-static methods....8
Array handling....10
Conditional Nested If Statements....11
Multiple selection statement....12
Complex Algorithm....13
Functions (parameterised, non-parameterised)....14
The constructor definition....15
Class definition....16
CRUD Operation....17
Create....17
Read....18
Update....20
Delete....22
Try & Catch Block....24

Imports and Dependencies

In my dev journey, I have used the least of the external libraries or frameworks that exist, but I have used those that are extremely required for my Java program to function effectively(Figure 1)



No	Jar File	Purpose/Use
1	mysql-connector-j-9.3.0.jar	JDBC driver for connecting Java applications to MySQL databases.
2	jbcrypt-0.4.jar	implementation of the BCrypt password hashing function. It's commonly used to secure and verify passwords
3	JDK 17	powers backend development in full-stack apps with a modern, stable Java platform.

Techniques complexity used

- Normal Framework
- Password encryption (hash salt)
- Import multiple packages based on the requirements of the project, Images Package
- Image handling
- Static and non-static methods
- Array handling
- Conditional statement nested if
- Iteration (for loop, while loop, nested loop)
- Complex Algorithm
- Functions (parameterised, non-parameterised)
- The constructor definition
- Class definition
- Database File handling (CRUD operation)

Normal Framework

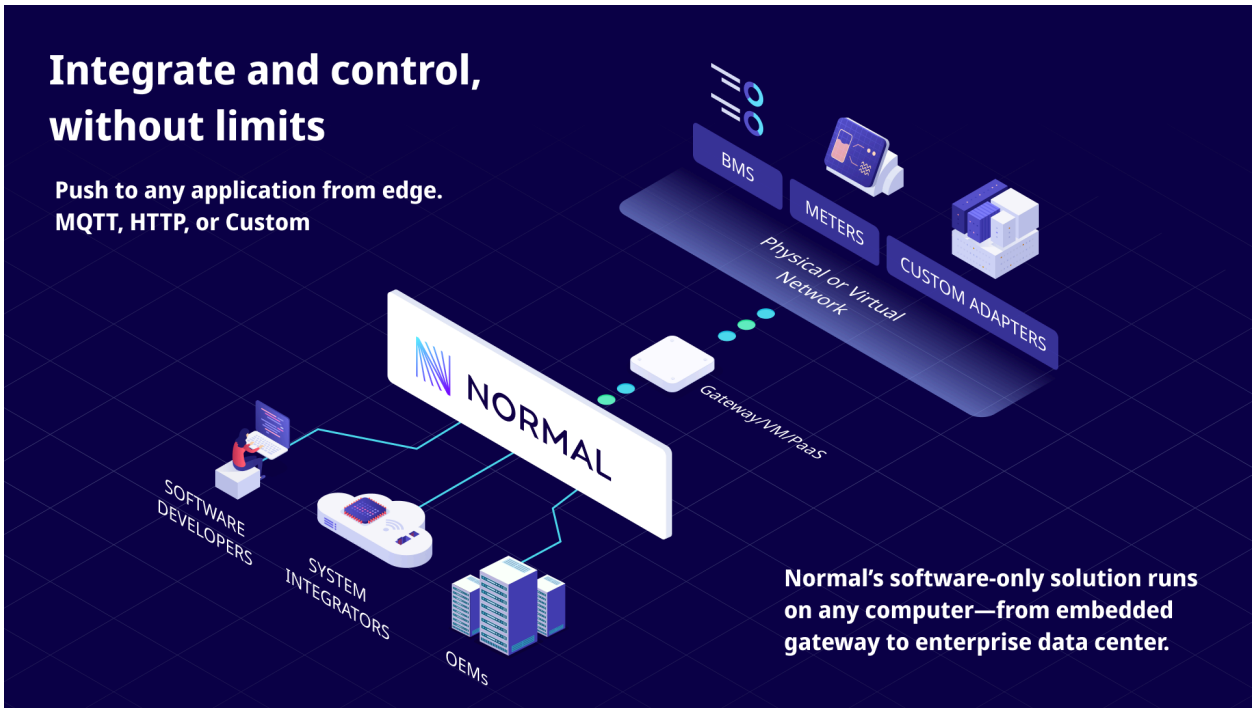
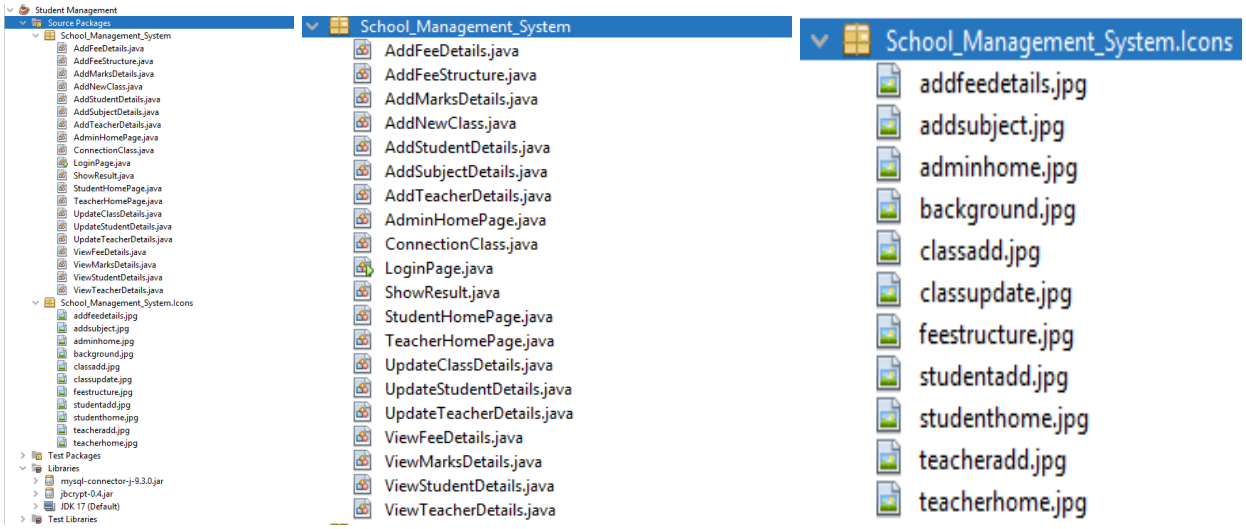


Image Credit: Normal Framework

The Normal Framework (NF) is a framework for software designed for Internet of Things (IoT) application developers, especially those working with systems. NF is built on years of experience in building, deploying, and operating cloud-connected software products.

Full breakdown of the files in the project



The Project's J-Package has Visual, Model and Data Control all together compressed in every single class, with ConnectionClass.java being a file for easy instantiation of SQL Connection Objects for each Class. Images J-Package has all the Background Images used for this project

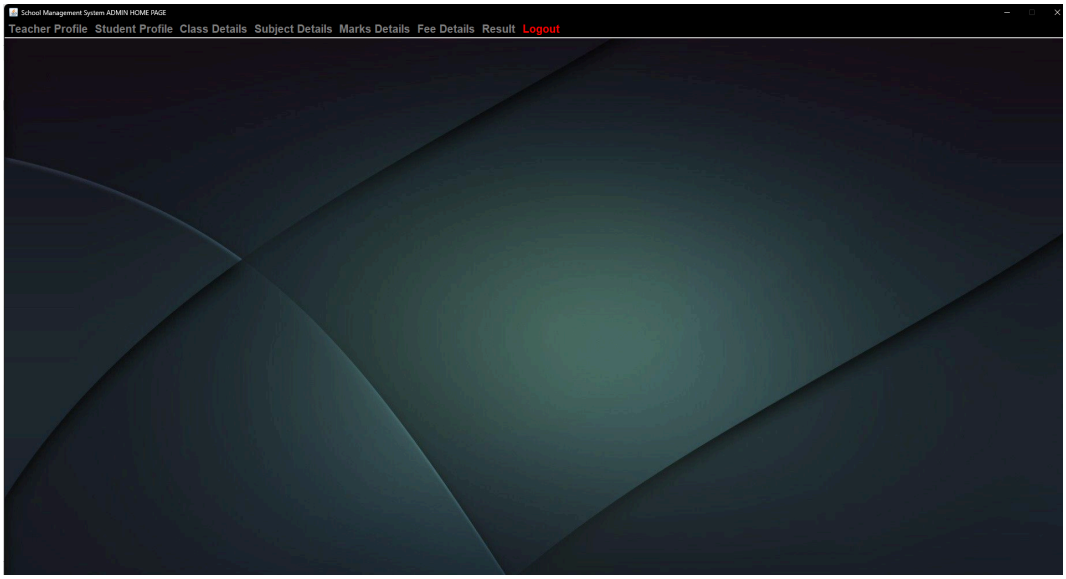
Screenshot from the AdminHomePage.java and AddSubjectDetails.java class showing the view capabilities

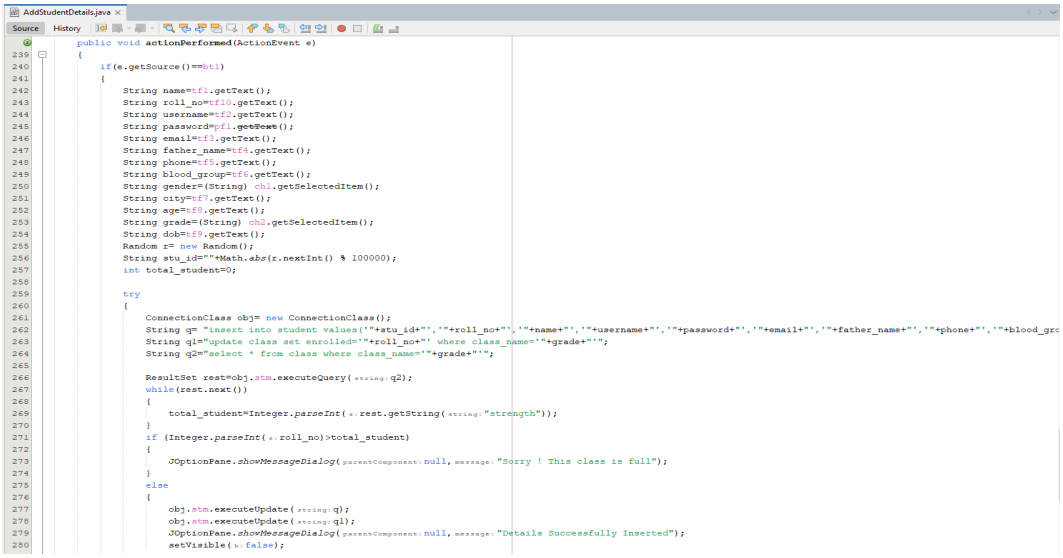
AdminHomePage.java

```
JMenuBar ml = new JMenuBar();
ml.setBackground(bg:Color.black.darker());

//menu buttons extender
JMenu menu1 = new JMenu(s:"Teacher Profile");
JMenu menu2 = new JMenu(s:"Student Profile");
JMenu menu3 = new JMenu(s:"Class Details");
JMenu menu4 = new JMenu(s:"Subject Details");
JMenu menu5 = new JMenu(s:"Marks Details");
JMenu menu6 = new JMenu(s:"Fee Details");
JMenu menu7 = new JMenu(s:"Result");
JMenu menu8 = new JMenu(s:"Logout");

// teacher menu items
JMenuItem ment1 = new JMenuItem(text:"Add Teacher Details");
JMenuItem ment2 = new JMenuItem(text:"Update Teacher Details");
JMenuItem ment3 = new JMenuItem(text:"View Teacher Details");

// student menu items
JMenuItem ment4 = new JMenuItem(text:"Add Student Details");
JMenuItem ment5 = new JMenuItem(text:"Update Student Details");
JMenuItem ment6 = new JMenuItem(text:"View Student Details");

// class menu items
JMenuItem ment7 = new JMenuItem(text:"Add Class Details");
JMenuItem ment8 = new JMenuItem(text:"Update Class Details");


// subject menu items
JMenuItem ment9 = new JMenuItem(text:"Add Subject Details");

// marks mark items
JMenuItem ment10 = new JMenuItem(text:"Add Marks");
JMenuItem ment11 = new JMenuItem(text:"View Marks");

// fee menu items
JMenuItem ment12 = new JMenuItem(text:"Add Fee Details");
JMenuItem ment13 = new JMenuItem(text:"Add Fee Structure");
JMenuItem ment14 = new JMenuItem(text:"View Fee Details");

// result menu items
JMenuItem ment15 = new JMenuItem(text:"Show Results");
```

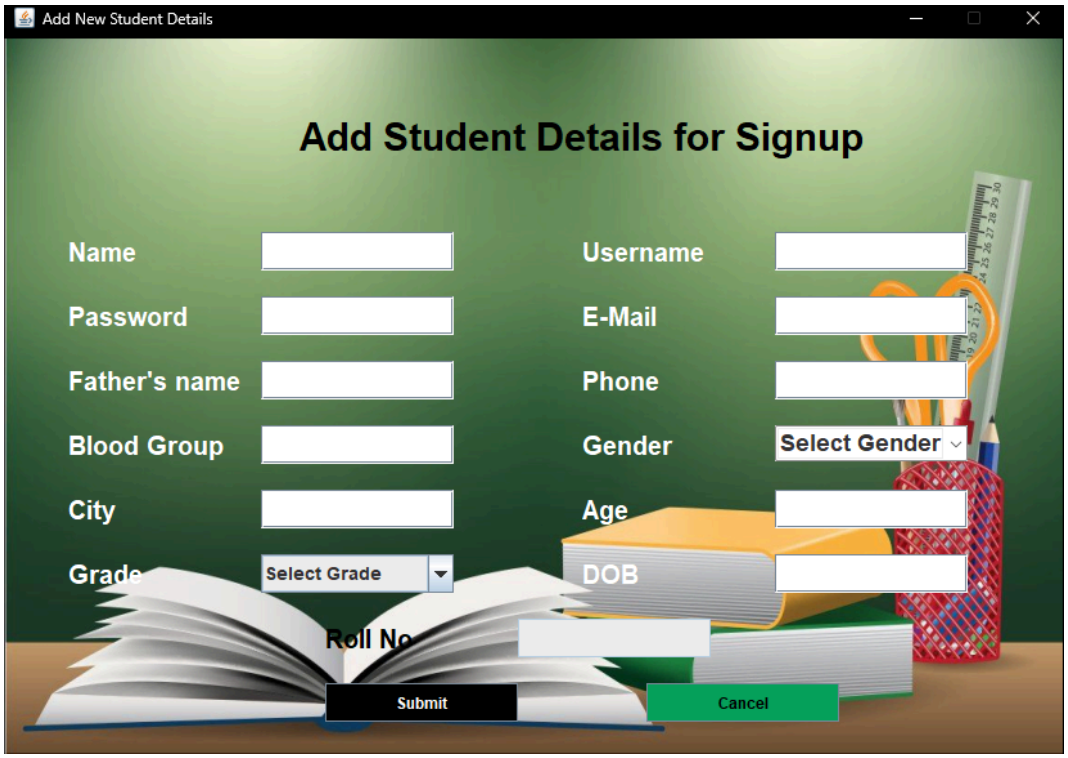
AdminHomePage.java

```
@Override
public void actionPerformed(ActionEvent ae)
{
    String comnd=ae.getActionCommand();
    if (comnd.equals(anObject:"Add Teacher Details"))
    {
//          System.out.println("Add Teacher Details class open");
        new AddTeacherDetails().setVisible(b:true);
    }
    else if (comnd.equals(anObject:"Update Teacher Details"))
    {
//          System.out.println("Update Teacher Details class open");
        new UpdateTeacherDetails(account:account2, username:pub_username).setVisible(b:true);
    }
    else if (comnd.equals(anObject:"View Teacher Details"))
    {
//          System.out.println("View Teacher Details class open");
        new ViewTeacherDetails(account:account2, username:pub_username).setVisible(b:true);
    }
    else if (comnd.equals(anObject:"Add Student Details"))
    {
//          System.out.println("Add Student Details class open");
        new AddStudentDetails().setVisible(b:true);
    }
    else if (comnd.equals(anObject:"Update Student Details"))
    {
//          System.out.println("Update Student Details class open");
        new UpdateStudentDetails(account:account2, username:pub_username).setVisible(b:true);
    }
    else if (comnd.equals(anObject:"View Student Details"))
    {
//          System.out.println("View Student Details class open");
        new ViewStudentDetails(pub_username:account2, account2:pub_username).setVisible(b:true);
    }
    else if (comnd.equals(anObject:"Add Class Details"))
    {
//          System.out.println("Add Class Details class open");
        new AddNewClass().setVisible(b:true);
    }
    else if (comnd.equals(anObject:"Update Class Details"))
    {
//          System.out.println("Update Class Details class open");
        new UpdateClassDetails().setVisible(b:true);
    }
```





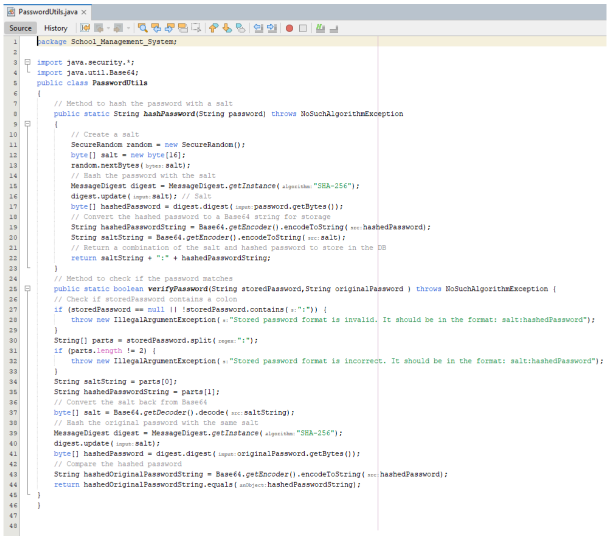
```
AddStudentDetails.java
Source  History
    public void actionPerformed(ActionEvent e)
    {
        if(e.getSource()==bt1)
        {
            String name=tf1.getText();
            String roll_no=tf10.getText();
            String username=tf2.getText();
            String password=pf1.getText();
            String email=tf3.getText();
            String father_name=tf4.getText();
            String phone=tf5.getText();
            String blood_group=tf6.getText();
            String gender=(String) ch1.getSelectedItem();
            String city=tf7.getText();
            String age=tf8.getText();
            String grade=(String) ch2.getSelectedItem();
            String dob=tf9.getText();
            Random r= new Random();
            String stu_id=""+Math.abs(r.nextInt() % 100000);
            int total_student=0;

            try
            {
                ConnectionClass obj= new ConnectionClass();
                String q= "insert into student values('"+stu_id+"','"+roll_no+"','"+name+"','"+username+"','"+password+"','"+email+"','"+father_name+"','"+phone+"','"+blood_gro
                String q1="update class set enrolled='"+roll_no+"' where class_name='"+grade+"'";
                String q2="select * from class where class_name='"+grade+"'";

                ResultSet rest=obj.stm.executeQuery( string: q2);
                while(rest.next())
                {
                    total_student=Integer.parseInt( s: rest.getString( string: "strength"));
                }
                if (Integer.parseInt( s: roll_no)>total_student)
                {
                    JOptionPane.showMessageDialog( parentComponent: null, message: "Sorry ! This class is full");
                }
                else
                {
                    obj.stm.executeUpdate( string: q);
                    obj.stm.executeUpdate( string: q1);
                    JOptionPane.showMessageDialog( parentComponent: null, message: "Details Successfully Inserted");
                    setVisible( b: false);
```

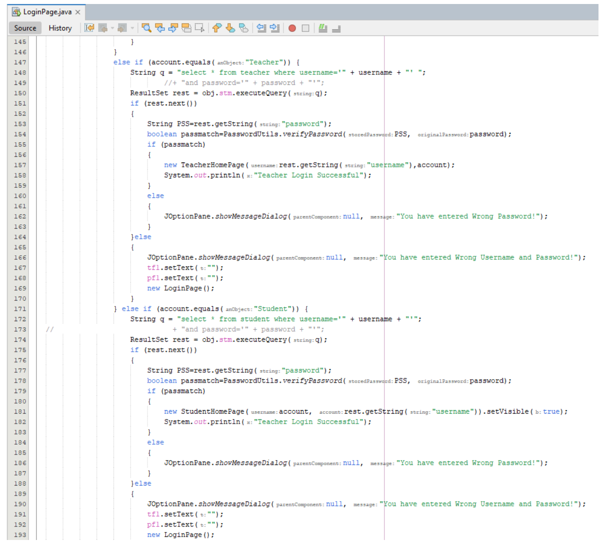


This framework helps keep all the parts together: the view, action listener and database connection for each respective section in its own file, also with the ConnectionClass.java file helps create new objects in different classes to get the connection to the database easily connected and working. This may consume a lot more storage at once but keeps everything easily known and organised for the main coder to know what and where is the code.

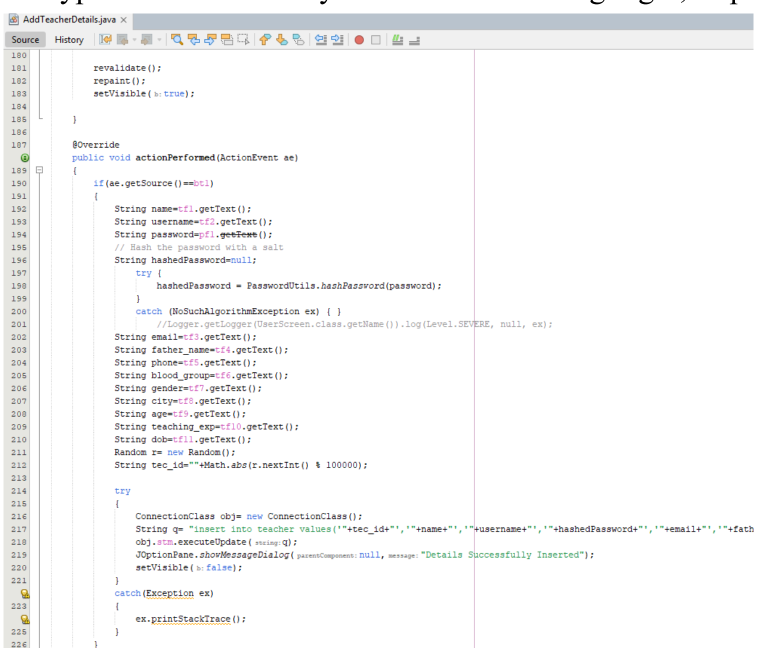
Password Encryption

Password Encryption is made possible with the use of the hash salt method in PasswordUtils.java





Encryption is successfully authenticated during login, as per the screenshot



Screenshot from addteacherdetails.java showing encryption during user creation

mysql> select * from teacher;

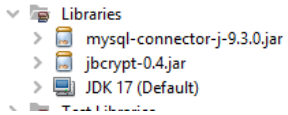
tec_id	name	username	password	email	father_name	phone	blood_group	gender	city	age	teaching_exp	dob
14774	Parth	Parth	XaLThj7H0SIoWQOKc2kviA==:VSsInFVohyCXajexfL39QVKVYrGBKQu93uotllBOQMs=	example@gamil.com	NA	111111111	O	Male	Some	Some	Some	Something
59016	Dharshan	teacher1	LVJk2Tb6SPSHccg3fxfw6A==:AlkpeuDYqf8TloEttPtvuJ0P2o0ynsrH4KCgTUNRvgE=	dharshan.j@gmail.com	Mr. J	1111111111	O+	Male	Kolhapur	30	10	1/1/1995
90921	Mahi Kumari	teacher2	5/dljOBrjvjF5/ygRX5Piw==:Ye4ya1+gtM6Yoc0P7MPfmCeIkcBtSX04TF593/AjaTw=	mahi.kumari@gmail.com	Mr Kumari	2222222222	O+	Female	Raichur	28	5	1/1/1997

Screenshot of database depicting the encryption of the password successfully, and the encrypted data is successfully inserted into the database for data security, protecting from any data breaches through X-SS.

Password is encrypted during user creation for password encryption and authentication to work successfully during login and data is encrypted.

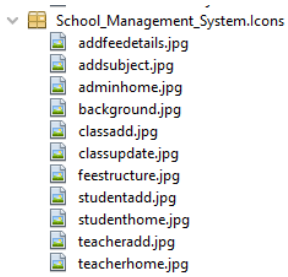
Import multiple packages

Libraries



Use of different Libraries to facilitate for the need of mysql connection through mysql-connector and jbcrypt for encryption procedures, these all have been made based on the requirements of the project.

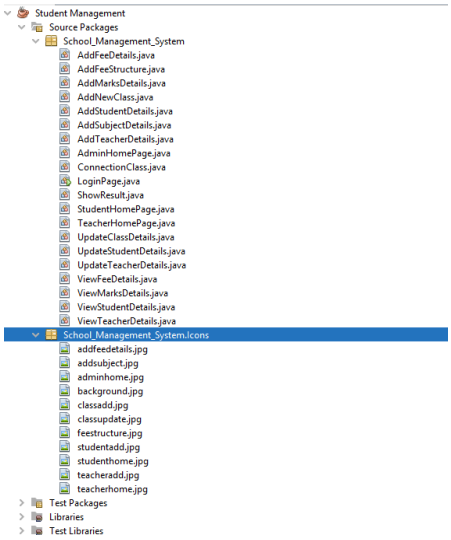
Images Package



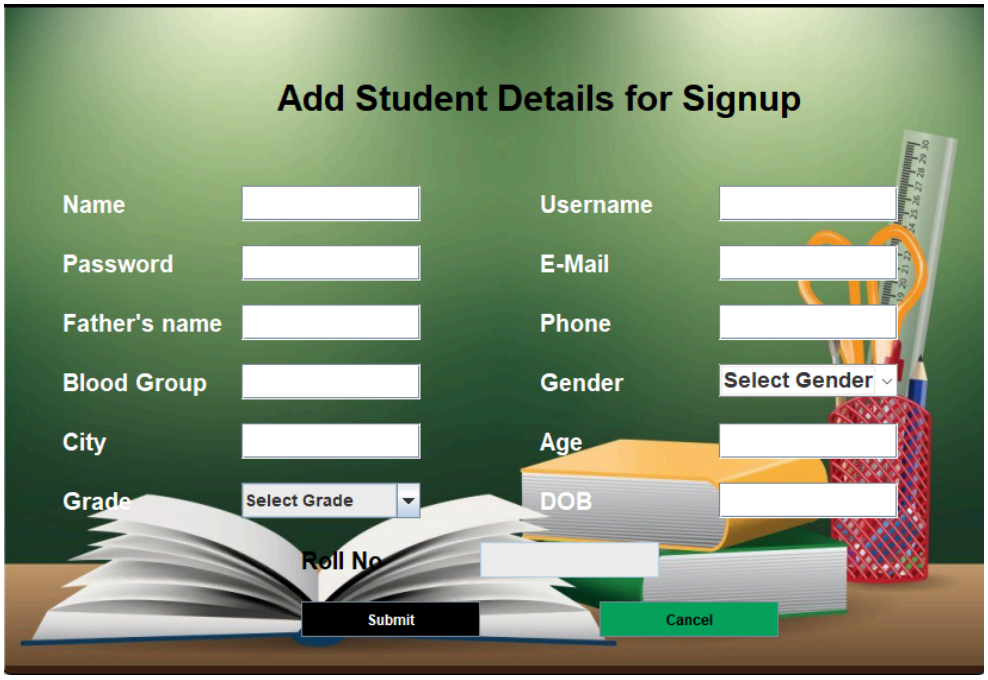
As per the requirement of the project, the use of different images have been done through the internet by inserting them with the project files in the Images Folder/package named as School_Management_System.Icons

Image handling

Holds data of all the images for the software, mostly background images which are found from the internet




```
28          // Load background
29          ImageIcon img = new ImageIcon( location: getClass().getResource( name: "Icons/studentadd.jpg"));
30          Image scaledImg = img.getImage().getScaledInstance( width: 840,  height: 600,  hints: Image.SCALE_SMOOTH);
31          JLabel bgl = new JLabel(new ImageIcon( image: scaledImg));
32          bgl.setBounds( x: 0,  y: 0,  width: 840,  height: 600);
33          bgl.setLayout( mgr: null);
34          add( comp: bgl);
```



Screenshot from AddStudentDetails, showing the Image handling using an image from the Product files, and above is the code showing the handling and mechanism of the whole process, of how the image is located and then manipulated to the requirements and used.

Static & Non-static methods

```java
package School_Management_System;

import java.security.*;
import java.util.Base64;
public class PasswordUtils
{
    // Method to hash the password with a salt
    public static String hashPassword(String password) throws NoSuchAlgorithmException
    {
        // Create a salt
        SecureRandom random = new SecureRandom();
        byte[] salt = new byte[16];
        random.nextBytes( bytes: salt);
        // Hash the password with the salt
        MessageDigest digest = MessageDigest.getInstance( algorithm: "SHA-256");
        digest.update( input: salt); // Salt
        byte[] hashedPassword = digest.digest( input: password.getBytes());
        // Convert the hashed password to a Base64 string for storage
        String hashedPasswordString = Base64.getEncoder().encodeToString( src: hashedPassword);
        String saltString = Base64.getEncoder().encodeToString( src: salt);
        // Return a combination of the salt and hashed password to store in the DB
        return saltString + ":" + hashedPasswordString;
    }
```

ViewFeeDetails.java × ConnectionClass.java ×

Source History

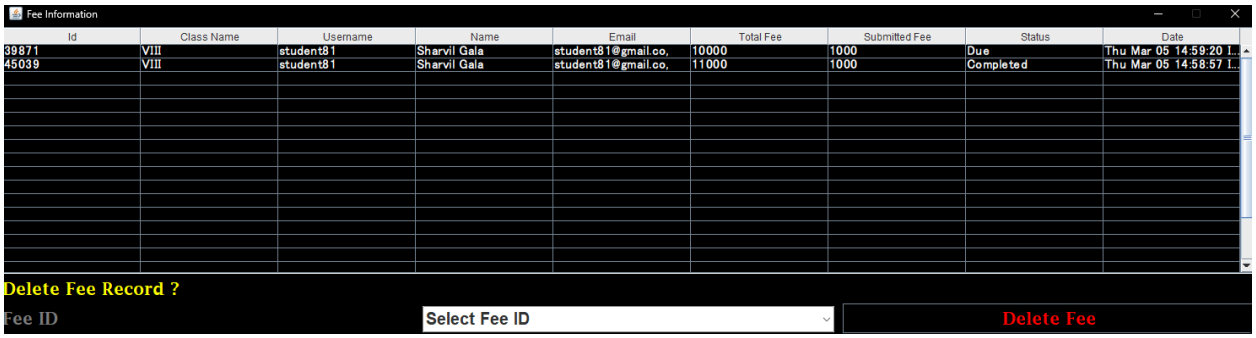
```java
public class ConnectionClass {
    public Connection con;
    public Statement stm;

    ConnectionClass() {
        try {
            // Load MySQL JDBC Driver
            Class.forName( className: "com.mysql.cj.jdbc.Driver");

            // Define connection URL
            String url = "jdbc:mysql://localhost:330/sms";
            String user = "root";
            String password = ""; // used "" as no password

            // Establish connection
            con = DriverManager.getConnection(url, user, password);
            stm = con.createStatement();

            if (con != null && !con.isClosed()) {
                System.out.println( x: "Connected to Database");
            } else {
                System.out.println( x: "Connection Failed");
            }
        } catch (ClassNotFoundException | SQLException ex) {
```

My product uses both static and non-static methods in using variables as parameters in respective classes, it follows a partial version of the Singleton Design pattern while it varies it alone does all the tasks of the connection to the database, hence it can be called as Singleton-style Database Connection Manager design.

Array handling

```
public class ViewFeeDetails extends JFrame
{
    String x[]={"Id", "Class Name", "Username", "Name", "Email", "Total Fee", "Submitted Fee", "Status", "Date"};
    String y[][]=new String [21][9];
    int i=0,j=0;
```

```
            ResultSet rest=obj.stm.executeQuery( string:query);
            while(rest.next())
            {
                y[i][j++]=rest.getString( string:"fee_id");
                y[i][j++]=rest.getString( string:"class_name");
                y[i][j++]=rest.getString( string:"username");
                y[i][j++]=rest.getString( string:"name");
                y[i][j++]=rest.getString( string:"email");
                y[i][j++]=rest.getString( string:"total_fee");
                y[i][j++]=rest.getString( string:"submited_fee");
                y[i][j++]=rest.getString( string:"status");
                y[i][j++]=rest.getString( string:"date");
                i++;
                j=0;
            }
```




```
mysql> select * from student_fee;
+--------+------------+-----------+--------------+--------------------+-----------+--------------+-----------+-----------------------------+
| fee_id | class_name | username  | name         | email              | total_fee | submited_fee | status    | date                        |
+--------+------------+-----------+--------------+--------------------+-----------+--------------+-----------+-----------------------------+
| 39871  | VIII       | student81 | Sharvil Gala | student81@gmail.co, | 10000     | 1000         | Due       | Thu Mar 05 14:59:20 IST 2026 |
| 45039  | VIII       | student81 | Sharvil Gala | student81@gmail.co, | 11000     | 1000         | Completed | Thu Mar 05 14:58:57 IST 2026 |
+--------+------------+-----------+--------------+--------------------+-----------+--------------+-----------+-----------------------------+
2 rows in set (0.01 sec)
```

Array is handled in View Classes, the above is a snippet from ViewFeeDetails, showing how the array is used for representing data from the database, and then the data is individually implemented to showcase a database type of data in the product.

Conditional Nested If Statements

Nested If Statement

During the product development phase, I have used nested if statements to open the correct panel based on what is called to open the correct menu of the project based on the command called and what needs to be run and opened given in that code. This also shows a variation of polymorphism.

```
AdminHomePage.java
Source  History
222     @Override
        public void actionPerformed(ActionEvent ae)
224     {
225         String commd=ae.getActionCommand();
226         if (commd.equals( anObject: "Add Teacher Details"))
227         {
228 //          System.out.println("Add Teacher Details class open");
229             new AddTeacherDetails().setVisible( b: true);
230         }
231         else if (commd.equals( anObject: "Update Teacher Details"))
232         {
233 //          System.out.println("Update Teacher Details class open");
234             new UpdateTeacherDetails( account: account2,  username: pub_username).setVisible( b: true);
235         }
236         else if (commd.equals( anObject: "View Teacher Details"))
237         {
238 //          System.out.println("View Teacher Details class open");
239             new ViewTeacherDetails( account: account2,  username: pub_username).setVisible( b: true);
240         }
241         else if (commd.equals( anObject: "Add Student Details"))
242         {
243 //          System.out.println("Add Student Details class open");
244             new AddStudentDetails().setVisible( b: true);
245         }
246         else if (commd.equals( anObject: "Update Student Details"))
247         {
248 //          System.out.println("Update Student Details class open");
249             new UpdateStudentDetails( account: account2,  username: pub_username).setVisible( b: true);
250         }
251         else if (commd.equals( anObject: "View Student Details"))
252         {
253 //          System.out.println("View Student Details class open");
254             new ViewStudentDetails( pub_username: account2,  account2: pub_username).setVisible( b: true);
255         }
256         else if (commd.equals( anObject: "Add Class Details"))
257         {
258 //          System.out.println("Add Class Details class open");
259             new AddNewClass().setVisible( b: true);
260         }
```

```
261         else if (commd.equals( anObject: "Update Class Details"))
262         {
263 //          System.out.println("Update Class Details class open");
264             new UpdateClassDetails().setVisible( b: true);
265         }
266         else if (commd.equals( anObject: "Add Subject Details"))
267         {
268 //          System.out.println("Add Subject Details class open");
269             new AddSubjectDetails().setVisible( b: true);
270         }
271         else if (commd.equals( anObject: "Add Marks"))
272         {
273 //          System.out.println("Add Mark Subject class open");
274             new AddMarksDetails().setVisible( b: true);
275         }
276         else if (commd.equals( anObject: "View Marks"))
277         {
278 //          System.out.println("Add Mark Subject class open");
279             new ViewMarksDetails(pub_username, account2).setVisible( b: true);
280         }
281         else if (commd.equals( anObject: "Add Fee Details"))
282         {
283 //          System.out.println("Add Fee Details class open");
284             new AddFeeDetails().setVisible( b: true);
285         }
286         else if (commd.equals( anObject: "Add Fee Structure"))
287         {
288             System.out.println( x: "Add Fee Structure class open");
289             new AddFeeStructure().setVisible( b: true);
290         }
291         else if (commd.equals( anObject: "View Fee Details"))
292         {
293             System.out.println( x: "View Fee Details class open");
294             new ViewFeeDetails(pub_username, account2).setVisible( b: true);
295         }
296         else if (commd.equals( anObject: "Show Results"))
297         {
298             System.out.println( x: "Show Results class open");
299             new ShowResult(pub_username, account2).setVisible( b: true);
300         }
```

```
301         else if (commd.equals( anObject: "Exit"))
302         {
303             System.out.println( x: "You are Logged Out");
304             this.setVisible( b: false);
305             new LoginPage();
306         }
307     }
```

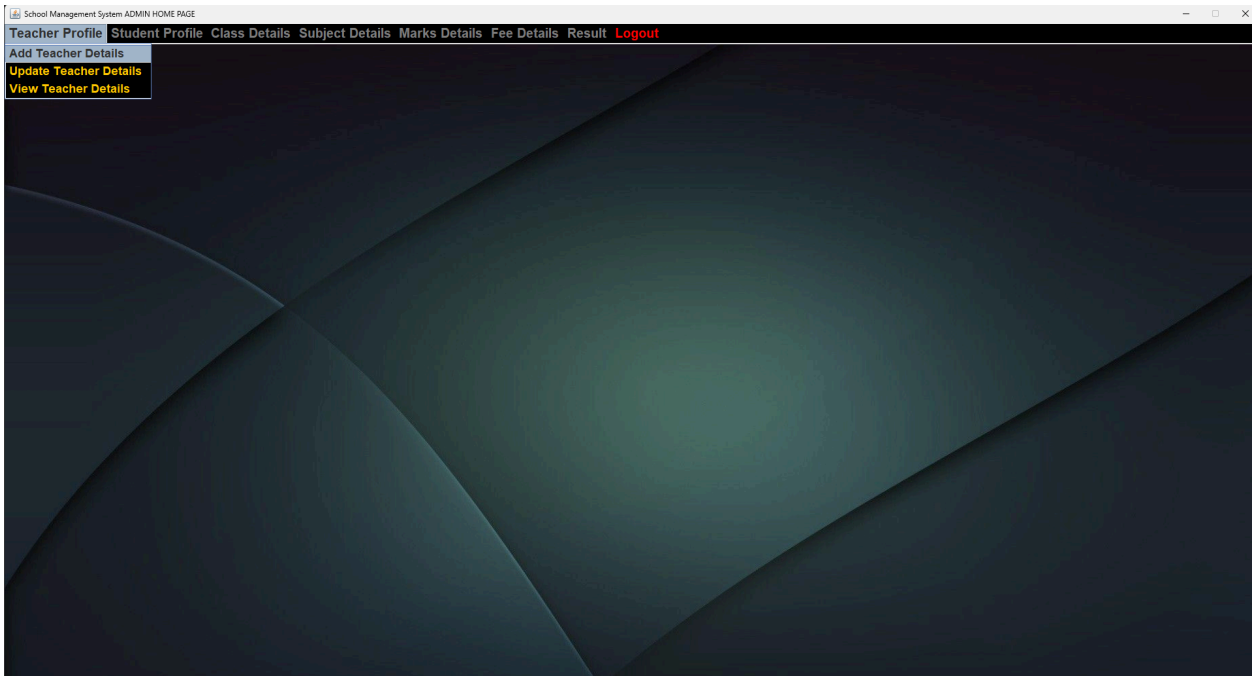
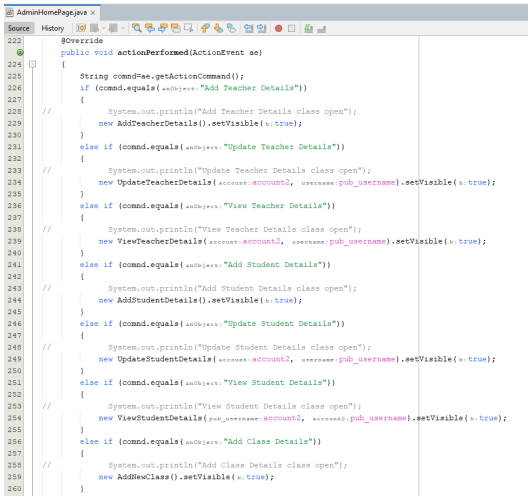
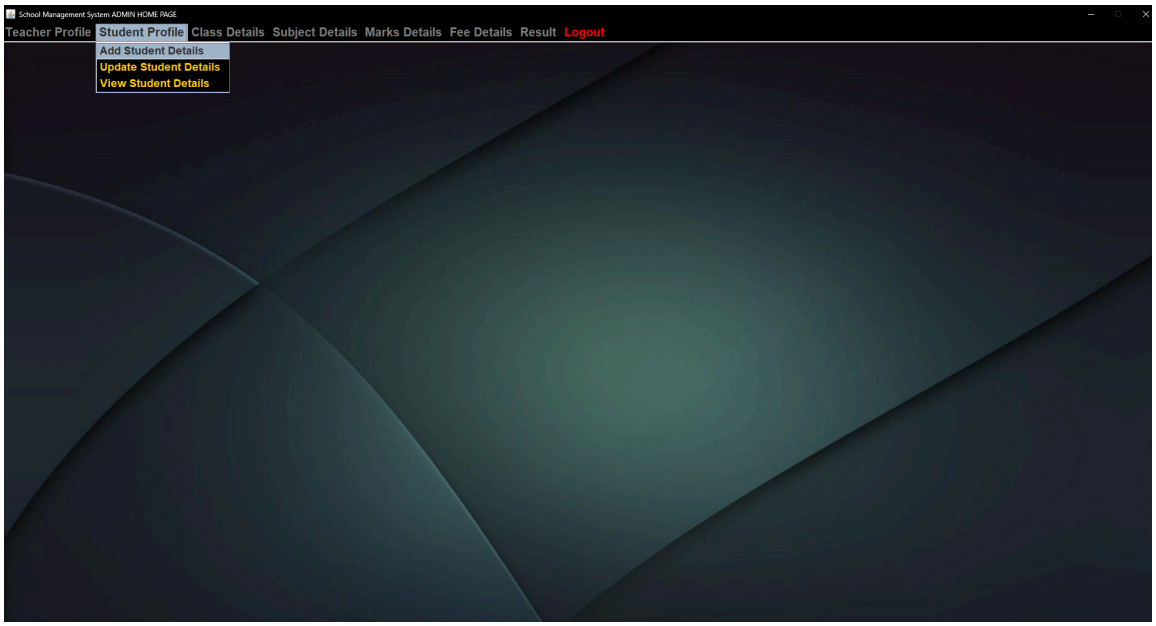


Image: Admin view of the Menu Panels available to view

Multiple selection statement

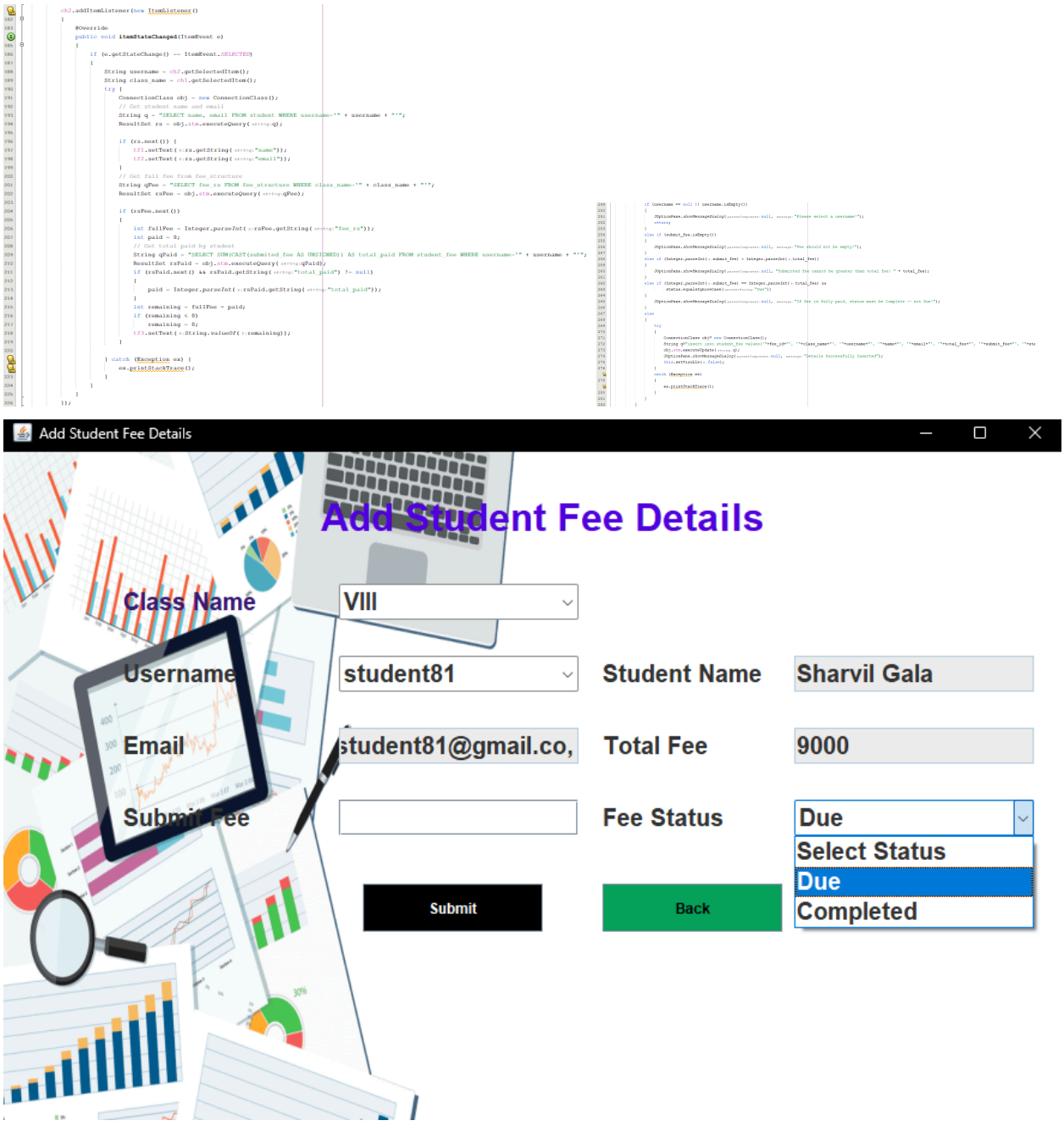


```
            else if (comnd.equals( anObject: "Exit"))
            {
                System.out.println( x: "You are Logged Out");
                this.setVisible( b: false);
                new LoginPage();
            }
        }
```



Nested loops are used to work on in the process of iteration helping in repeatedly execution of a block of code until a certain condition is met or for a fixed number of times.

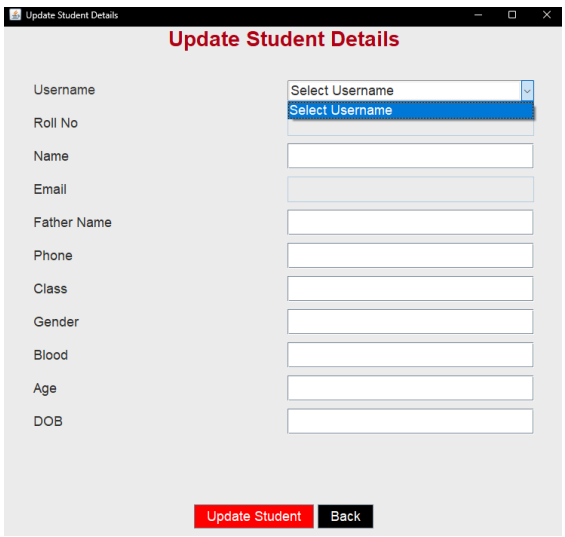
Complex Algorithm

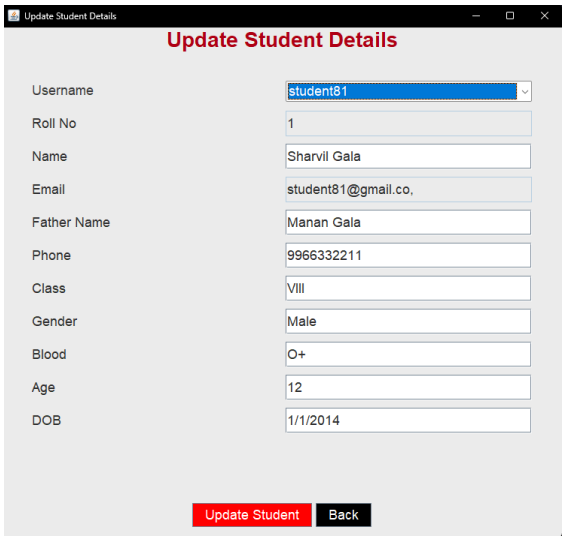


In complex algorithms, I am using my AddFeeDetails, fee calculation method which calculates the pending amount of fees when selected grade and username in Add Fee Details, UI it will calculate the total fee as per the fee structure minused fee data which have been inserted as completed and those which have been inserted as due will be added upon, whilst not adding past the fee structure limit for based on the student's class specification. Along with during data entry into the system the algorithm does not allow fees larger then fee structure to be added, nor accept the full amount of paid fees to be marked as due.

Functions (parameterised, non-parameterised)

```
245         else if (commd.equals( anObject: "Update Student Details"))
246         {
247 //              System.out.println("Update Student Details class open");
248             new UpdateStudentDetails( account: account2,  username: pub_username).setVisible( b: true);
249         }
250         else if (commd.equals( anObject: "View Student Details"))
251         {
252 //              System.out.println("View Student Details class open");
253             new ViewStudentDetails( pub_username: account2,  account2: pub_username).setVisible( b: true);
254         }
255         else if (commd.equals( anObject: "Add Class Details"))
256         {
257 //              System.out.println("Add Class Details class open");
258             new AddNewClass().setVisible( b: true);
259         }
```





For functions, from AdminHomePage, the parameters when opening certain System parts which have critical and points through which unauthorised access can be created such as Update Student Details, where it is blocked by parameters As shown in Image 1 of the panel showing unaccessibility for data safety and integrity and Image 2 is showing proper functionality when the parameters are acheived. While Class details which condole none to low harm do not have parameters and depict non parameterised functions.

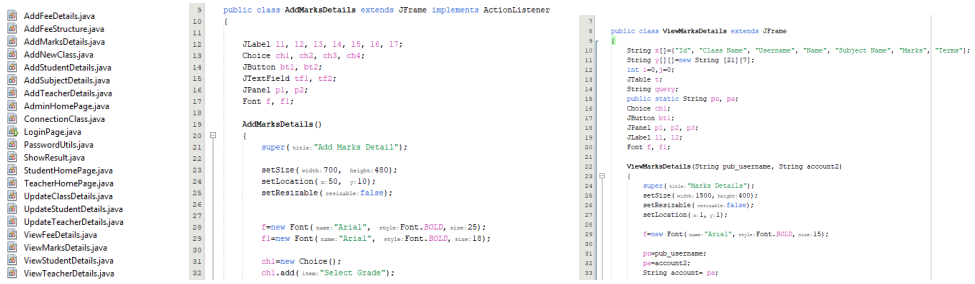
The constructor definition

```
7
8   public class UpdateStudentDetails extends JFrame implements ActionListener
9   {
10      JLabel l1, l2, l3, l4, l5, l6, l7, l8, l9, l10, l11, l12, l13, l14;
11      JPanel p1, p2, p3;
12      JTextField tf1, tf2, tf3, tf4, tf5, tf6, tf7, tf8, tf9, tf10, tf11;
13      Choice ch1;
14      JButton bt1, bt2;
15      public String pubu, account2;
16
17      UpdateStudentDetails(String account, String username)
18      {
19          super( title: "Update Student Details");
20          setSize( width: 760,  height: 720);
21          setLocation( x: 550, y: 200);
22          setVisible( b: true);
23          account2=account;
24          pubu=username;
25
26          Font f = new Font( name: "Arial",  style: Font.BOLD,  size: 28);
27          Font f1 = new Font( name: "Arial",  style: Font.PLAIN,  size: 18);
28
29          ch1=new Choice();
30          ch1.setFont( f: f1);
31          ch1.add( item: "Select Username");
```

```
245             else if (comnd.equals( anObject: "Update Student Details"))
246             {
247     //              System.out.println("Update Student Details class open");
248                 new UpdateStudentDetails( account: account2,  username: pub_username).setVisible( b: true);
249             }
```

In this constructor, UpdateStudentDetails, is used to initialise all the data members and update the data in the database with the new data value.

Class definition



In my project, the class acts as , AddMarksDetails and ViewMarksDetails, which extend JFrame to create GUI windows in a Student Management System. AddMarksDetails is used to input and store student marks, while ViewMarksDetails is used to retrieve and display marks in a JTable from the database.

CRUD Operation

Create

This CRUD operation in my product is a minimal requirement of the client. Each of the components (User, student, teacher, subject etc) have CRUD operation. The CRUD operation uses the Front-End(NetBeans IDE -swing components) and Back-End(SQL Database) mechanism to store data efficiently .

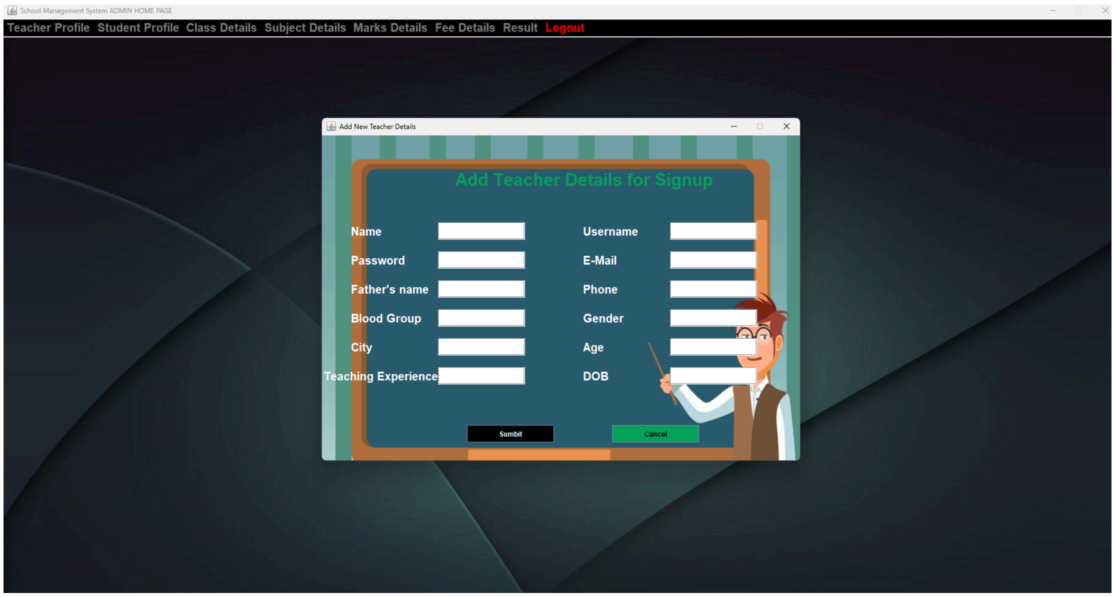


Image: Admin view of Teacher profile creation

```java
@Override
public void actionPerformed(ActionEvent ae)
{
    if(ae.getSource()==bt1)
    {
        String name=tf1.getText();
        String username=tf2.getText();
        String password=pf1.getText();
        // Hash the password with a salt
        String hashedPassword=null;
            try {
                hashedPassword = PasswordUtils.hashPassword(password);
            }
            catch (NoSuchAlgorithmException ex) { }
                //Logger.getLogger(UserScreen.class.getName()).log(Level.SEVERE, null, ex);
        String email=tf3.getText();
        String father_name=tf4.getText();
        String phone=tf5.getText();
        String blood_group=tf6.getText();
        String gender=tf7.getText();
        String city=tf8.getText();
        String age=tf9.getText();
        String teaching_exp=tf10.getText();
        String dob=tf11.getText();
        Random r= new Random();
        String tec_id=""+Math.abs(r.nextInt() % 100000);

        try
        {
            ConnectionClass obj= new ConnectionClass();
            String q= "insert into teacher values('"+tec_id+"','"+name+"','"+username+"','"+hashedPassword+"','"+email+"','"+father_name
            obj.stm.executeUpdate(string: q);
            JOptionPane.showMessageDialog(parentComponent: null, message: "Details Successfully Inserted");
            setVisible(b: false);
        }
        catch(Exception ex)
        {
            ex.printStackTrace();
```

Code of TeacherProfile.java

mysql> select * from teacher;

tec_id	name	username	password	email	father_name	phone	blood_group	gender	city	age	teaching_exp	dob
59016	Dharshan	teacher1	LVJk2Tb6SPSHccg3fxfw6A==:AlkpeuDYqf8TloEttPtvuJ0P2o0ynsrH4KCgTUNRvgE=	dharshan.j@gmail.com	Mr. J	1111111111	O+	Male	Kolhapur	31	10	1/1/1995
90921	Mahi Kumari	teacher2	5/dljOBrjvjF5/ygRX5Piw==:Ye4ya1+gtM6Yoc0P7MPfmCeIkcBtSX04TF593/AjaTw=	mahi.kumari@gmail.com	Mr Kumari	2222222222	O+	Female	Raichur	28	5	1/1/1997

2 rows in set (0.01 sec)

Record in Database viewed from MySQL 8.0 Command Line Client (hashing is enabled, depict it's working during CRUD function's **CREATE** to be visible for the following screenshots)

Read

My projects also use the CRUD's Read Function in a way to view the details that already exist in the database

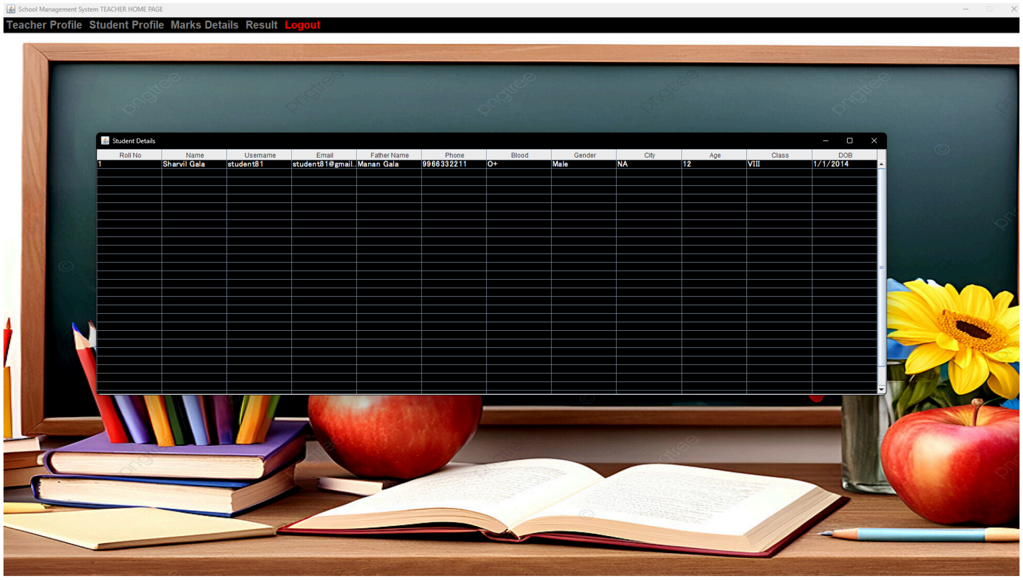


Image: Teacher view of ViewStudentDetails

```java
ViewStudentDetails(String pub_username, String account2)
{
    super("Student Details");
    setSize(1500, 400);
    setResizable(true);
    setLocation(1, 1);
    f=new Font("MS UI Gothic", Font.BOLD, 15);

    pa=pub_username;
    pu=account2;

    System.out.println(pu +  "pub_username" + "Account type :" + pa);
    try
    {
        ConnectionClass obj=new ConnectionClass();
        if(pa.equals("Student"))
        {
            query="select * from student where username='"+pu+"'";
        }
        else if(pa.equals("Admin"))
        {
            query="select * from student";
        }
        else if(pa.equals("Teacher"))
        {
            query="select * from student";
        }
        ResultSet rest=obj.stm.executeQuery(query);
        while(rest.next())
        {
            y[i][j++]=rest.getString("roll_no");
            y[i][j++]=rest.getString("name");
            y[i][j++]=rest.getString("username");
            y[i][j++]=rest.getString("email");
            y[i][j++]=rest.getString("father_name");
            y[i][j++]=rest.getString("phone");
            y[i][j++]=rest.getString("blood");
            y[i][j++]=rest.getString("gender");
            y[i][j++]=rest.getString("city");
            y[i][j++]=rest.getString("age");
            y[i][j++]=rest.getString("class");
            y[i][j++]=rest.getString("dob");
            i++;
            j=0;
```

Code from ViewStudentDetails.java

```
mysql> select * from student;
+--------+---------+------------------+----------+------------+---------------------------------+-------------+-----------+-------+--------+----------+-----+-------+----------+
| stu_id | roll_no | name             | username | password   | email                           | father_name | phone     | blood | gender | city     | age | class | dob      |
+--------+---------+------------------+----------+------------+---------------------------------+-------------+-----------+-------+--------+----------+-----+-------+----------+
| 1722   | 1       | Deshna Jain      | student1 | student123 | deshna.jain@sgischool.in        | Na          | 999999999 | O+    | Female | Kolhapur | 16  | IX    | 1/1/2008 |
| 41647  | 1       | Sukran Nirankari | student2 | student123 | sukran.nirankari@sgischool.in   | NA          | 999999999 | O+    | Male   | Kolhapur | 16  | IX    | 1/1/2008 |
| 98342  | 2       | Rishidatta       | student3 | student123 | rishidatta.shah@sgischool.in    | NA          | 666666666 | O+    | Male   | Nipani   | 16  | VIII  | 1/1/2008 |
+--------+---------+------------------+----------+------------+---------------------------------+-------------+-----------+-------+--------+----------+-----+-------+----------+
3 rows in set (0.00 sec)
```

Record in Database viewed from MySQL 8.0 Command Line Client (hashing was disabled for CRUD function's **READ** to be visible for following screesnhots)

Update

The project also uses CRUD's Update function to update details from within the program, which updates in the database in real time. Which also gets changed in the database as per what is needed.

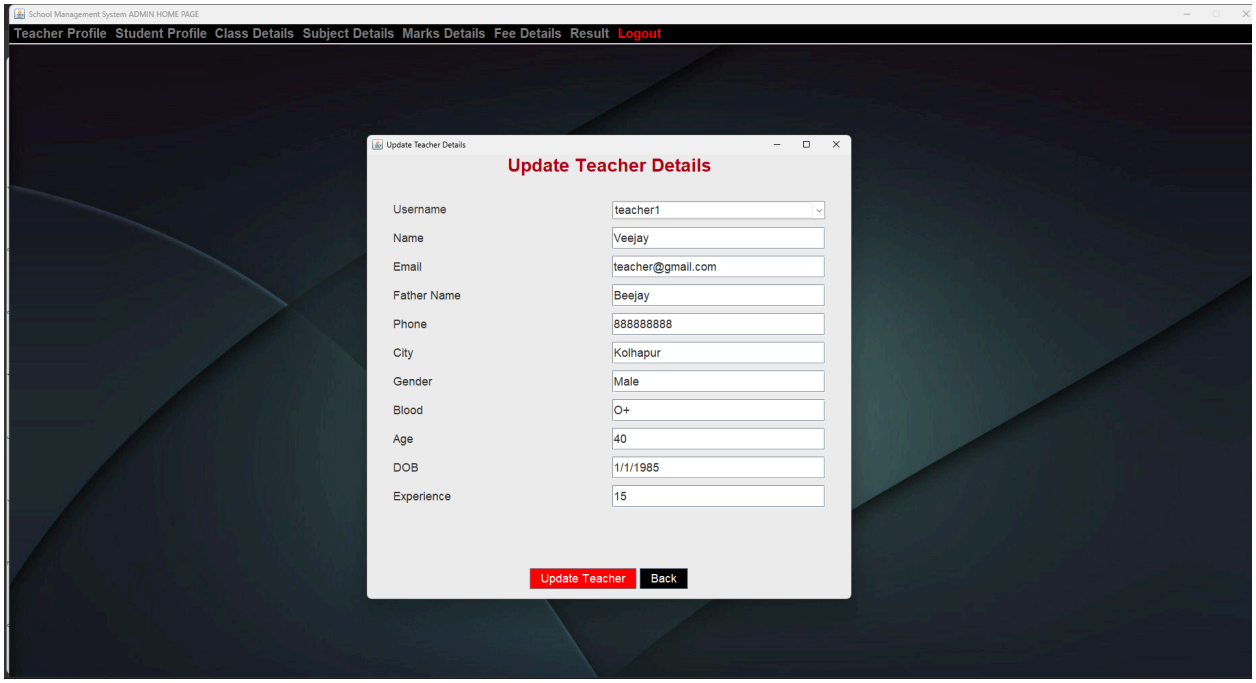


Image: Admin view of Update Teacher Details, before Updating in program

```
mysql> select * from teacher;
+--------+--------------+----------+------------+---------------------------+-------------+------------+-------------+--------+-----------+------+--------------+-----------+
| tec_id | name         | username | password   | email                     | father_name | phone      | blood_group | gender | city      | age  | teaching_exp | dob       |
+--------+--------------+----------+------------+---------------------------+-------------+------------+-------------+--------+-----------+------+--------------+-----------+
| 22695  | Veejay       | teacher1 | teacher123 | teacher@gmail.com         | Beejay      | 888888888  | O+          | Male   | Kolhapur  | 40   | 15           | 1/1/1985  |
| 75983  | Santanu Ghosh | teacher2 | teacher123 | santanu.ghosh@sgischool.in | NA          | 9999999999 | O+          | Male   | Kolkata   | 39   | 15           | 1/1/1986  |
+--------+--------------+----------+------------+---------------------------+-------------+------------+-------------+--------+-----------+------+--------------+-----------+
2 rows in set (0.00 sec)
```

Record in Database viewed from MySQL 8.0 Command Line Client, before update

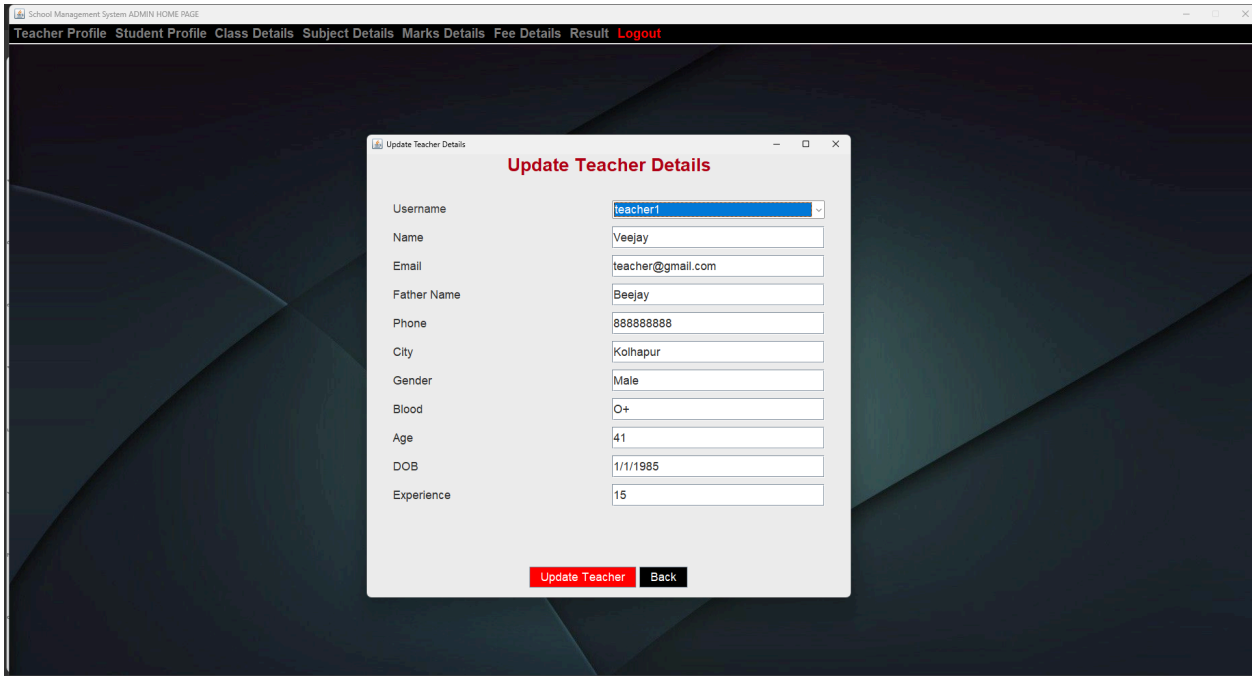
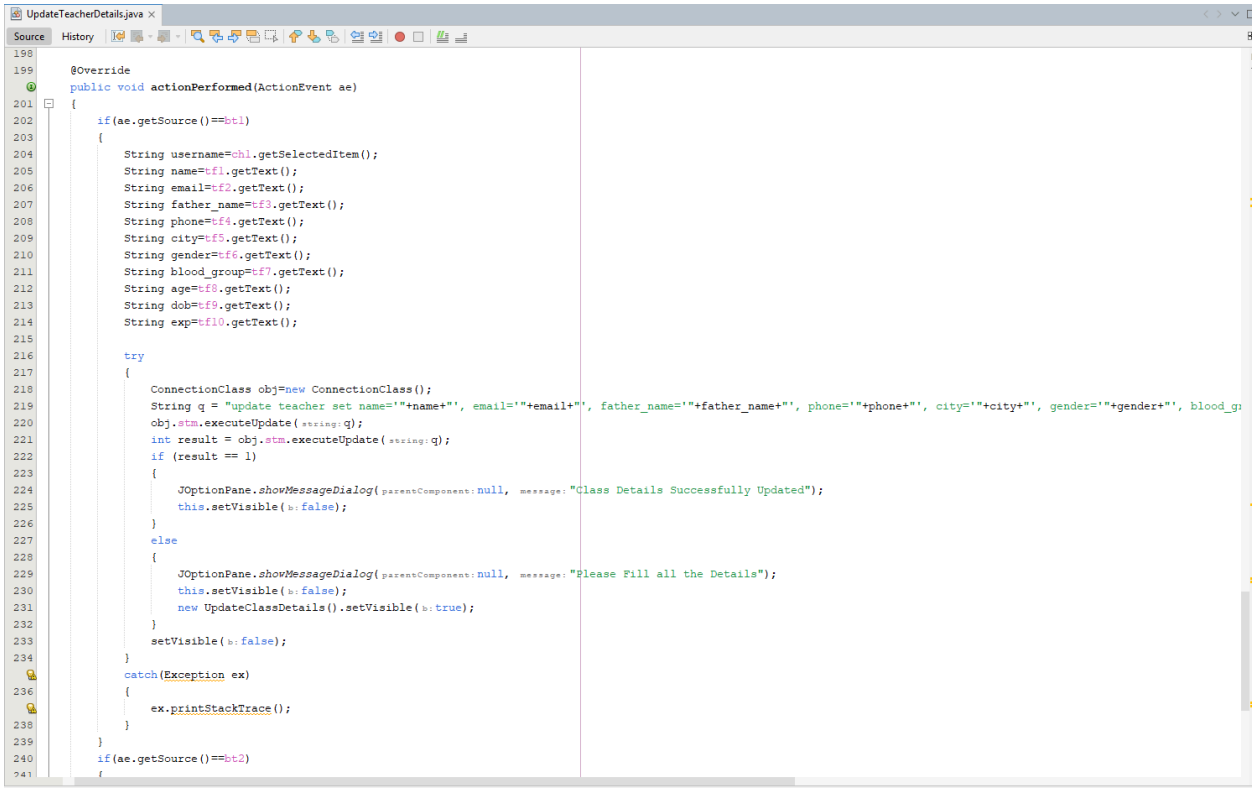


Image: Admin view of Update Teacher Details, after Updating in program (change age +1)

```
mysql> select * from teacher;
+--------+--------------+----------+------------+-------------------------+-------------+------------+-------------+--------+-----------+------+--------------+-----------+
| tec_id | name         | username | password   | email                   | father_name | phone      | blood_group | gender | city      | age  | teaching_exp | dob       |
+--------+--------------+----------+------------+-------------------------+-------------+------------+-------------+--------+-----------+------+--------------+-----------+
| 22695  | Veejay       | teacher1 | teacher123 | teacher@gmail.com       | Beejay      | 888888888  | O+          | Male   | Kolhapur  | 41   | 15           | 1/1/1985  |
| 75983  | Santanu Ghosh | teacher2 | teacher123 | santanu.ghosh@sgischool.in | NA       | 9999999999 | O+          | Male   | Kolkata   | 39   | 15           | 1/1/1986  |
+--------+--------------+----------+------------+-------------------------+-------------+------------+-------------+--------+-----------+------+--------------+-----------+
2 rows in set (0.00 sec)
```

Record in Database viewed from MySQL 8.0 Command Line Client, after update



```
@Override
public void actionPerformed(ActionEvent ae)
{
    if(ae.getSource()==bt1)
    {
        String username=ch1.getSelectedItem();
        String name=tf1.getText();
        String email=tf2.getText();
        String father_name=tf3.getText();
        String phone=tf4.getText();
        String city=tf5.getText();
        String gender=tf6.getText();
        String blood_group=tf7.getText();
        String age=tf8.getText();
        String dob=tf9.getText();
        String exp=tf10.getText();

        try
        {
            ConnectionClass obj=new ConnectionClass();
            String q = "update teacher set name='"+name+"', email='"+email+"', father_name='"+father_name+"', phone='"+phone+"', city='"+city+"', gender='"+gender+"', blood_g
            obj.stm.executeUpdate( string: q);
            int result = obj.stm.executeUpdate( string: q);
            if (result == 1)
            {
                JOptionPane.showMessageDialog( parentComponent: null,  message: "Class Details Successfully Updated");
                this.setVisible( b: false);
            }
            else
            {
                JOptionPane.showMessageDialog( parentComponent: null,  message: "Please Fill all the Details");
                this.setVisible( b: false);
                new UpdateClassDetails().setVisible( b: true);
            }
            setVisible( b: false);
        }
        catch(Exception ex)
        {
            ex.printStackTrace();
        }
    }
    if(ae.getSource()==bt2)
    {
```

Code from UpdateTeacherDetails.java

Delete

This function helps remove certain data from the database from within the application and works with instant changes

For this I have implemented it into the ViewTeacherDetails, for which I did not use it in the Read using Admin Account, instead used a Teacher Account with View Student Details as View Teacher Details has parameters to hide other teachers Details and Show only that accounts related details registered into the system.

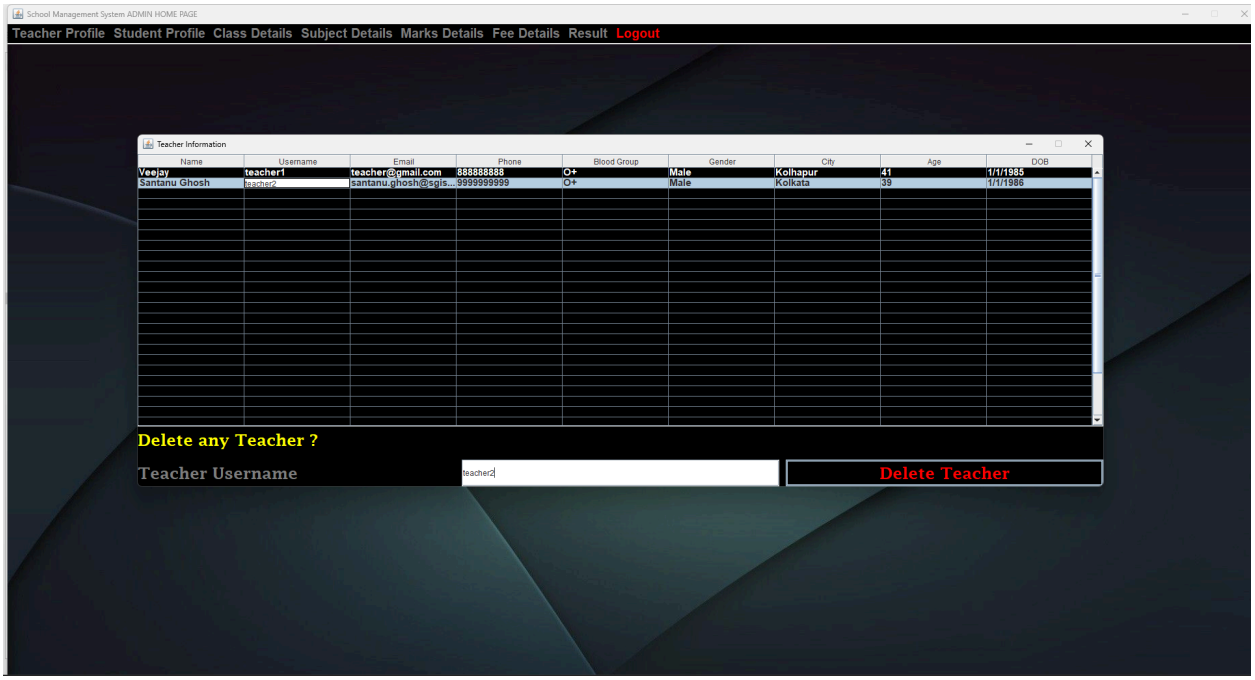


Image: Admin view of ViewTeacherDetails, Before Delete

```
mysql> select * from teacher;
+--------+--------------+----------+-----------+-------------------------+-------------+------------+-------------+--------+----------+------+--------------+------------+
| tec_id | name         | username | password  | email                   | father_name | phone      | blood_group | gender | city     | age  | teaching_exp | dob        |
+--------+--------------+----------+-----------+-------------------------+-------------+------------+-------------+--------+----------+------+--------------+------------+
| 22695  | Veejay       | teacher1 | teacher123 | teacher@gmail.com       | Beejay      | 888888888  | O+          | Male   | Kolhapur | 41   | 15           | 1/1/1985   |
| 75983  | Santanu Ghosh | teacher2 | teacher123 | santanu.ghosh@sgischool.in | NA       | 9999999999 | O+          | Male   | Kolkata  | 39   | 15           | 1/1/1986   |
+--------+--------------+----------+-----------+-------------------------+-------------+------------+-------------+--------+----------+------+--------------+------------+
2 rows in set (0.00 sec)
```

Record in Database viewed from MySQL 8.0 Command Line Client, before Delete

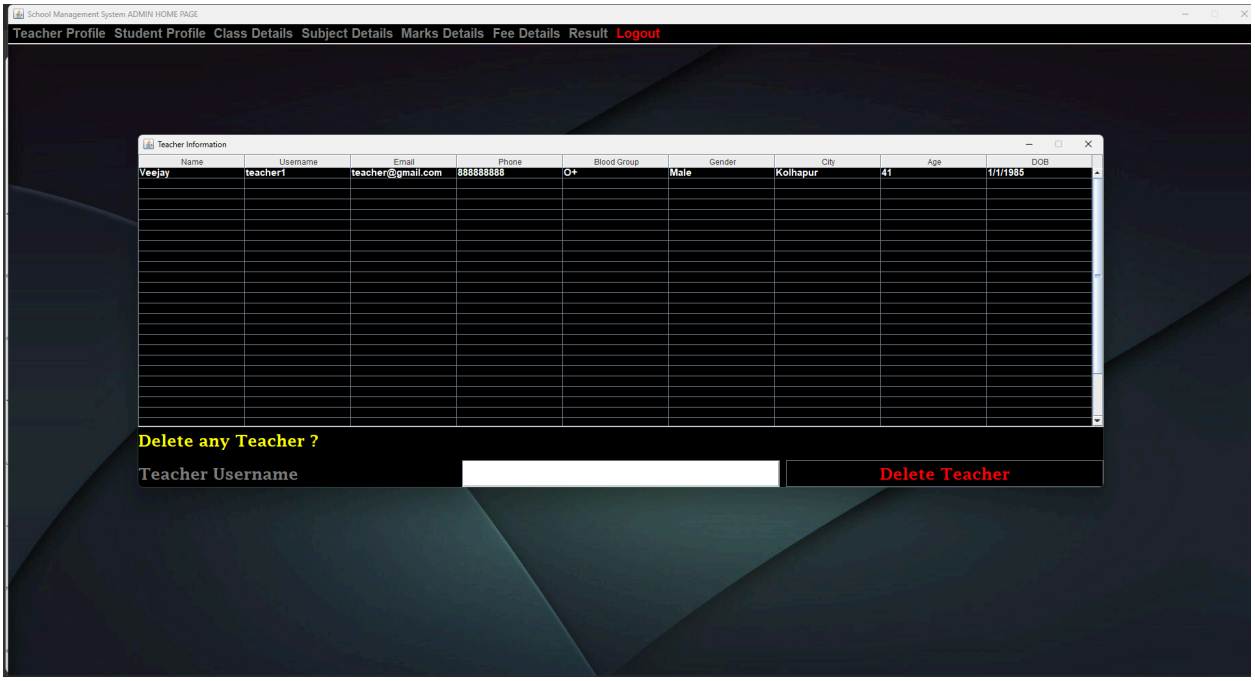


Image: Admin view of ViewTeacherDetails, After Delete

```
mysql> select * from teacher;
+--------+--------+----------+------------+-------------------+-------------+-----------+-------------+--------+----------+------+--------------+----------+
| tec_id | name   | username | password   | email             | father_name | phone     | blood_group | gender | city     | age  | teaching_exp | dob      |
+--------+--------+----------+------------+-------------------+-------------+-----------+-------------+--------+----------+------+--------------+----------+
| 22695  | Veejay | teacher1 | teacher123 | teacher@gmail.com | Beejay      | 888888888 | O+          | Male   | Kolhapur | 41   | 15           | 1/1/1985 |
+--------+--------+----------+------------+-------------------+-------------+-----------+-------------+--------+----------+------+--------------+----------+
1 row in set (0.00 sec)
```

Record in Database viewed from MySQL 8.0 Command Line Client, After Delete

ViewTeacherDetails.java

```
        }
        catch(Exception ex)
        {
            ex.printStackTrace();
        }
    }
    public void actionPerformed (ActionEvent e)
    {
        if(e.getSource()==btl)
        {

            String username=tfl.getText();
            try
            {
                ConnectionClass obj=new ConnectionClass();
                String q="delete from teacher where username='"+username+"'";
                int res=obj.stm.executeUpdate( string: q);
                if(res==1)
                {
                    JOptionPane.showMessageDialog( parentComponent: null,  message: "Teacher record is deleted !");
                    setVisible( b: false);
                }
                else
                {
                    JOptionPane.showMessageDialog( parentComponent: null,  message: "Teacher record does not match !");
                }
            }
            catch(Exception ex)
            {
                ex.printStackTrace();
            }
        }
    }

//    public static void main(String[] args)
//    {
//        new ViewTeacherDetails("account", "username").setVisible(true);
//    }
}
```

Code from ViewTeacherDetails.java

Try & Catch Block

In Java, the try and catch blocks is a special feature that helps to manage errors that happen while a program is running. An alternative to crashing when something goes wrong, this allows the program to catch the error and respond to it properly. This way, you can prevent unexpected failures and keep the program running smoothly. I have used these blocks to handle in each and every place in my developed software to handle runtime errors effectively and efficiently

```
try
{
    ConnectionClass obj=new ConnectionClass();
    String class_name=ch1.getSelectedItem();
    String username=ch2.getSelectedItem();

    String q="select * from marks where class_name='"+class_name+"' and username='"+username+"'";
    ResultSet rest1=obj.stm.executeQuery( string: q);

    while(rest1.next())
    {
        ta.append("\n\n Student Name : " +rest1.getString( string: "name"));
        ta.append("\n Div " +rest1.getString( string: "term"));
        ta.append( str: "\n ========================\n");
        ta.append("\n"+rest1.getString( string: "subject_name")+" : "+rest1.getString( string: "marks"));

        ta.append( str: "\n ========================\n");

        total=Integer.parseInt( s: rest1.getString( string: "marks"))+total;
        float div = (total * 100f) / 500f;
        ta.append("\nTotal : " +total);
        ta.append("\nDiv : " +div+"%");
    }
}
catch(Exception ex)
{
    ex.printStackTrace();
}
```

Code from the ShowResults.java file

Bibliography

- “Normal Framework.” *Normal.dev*, 2020, docs2.normal.dev/. Accessed 1 Sept. 2025.

Word Count Criteria C: 1402 words